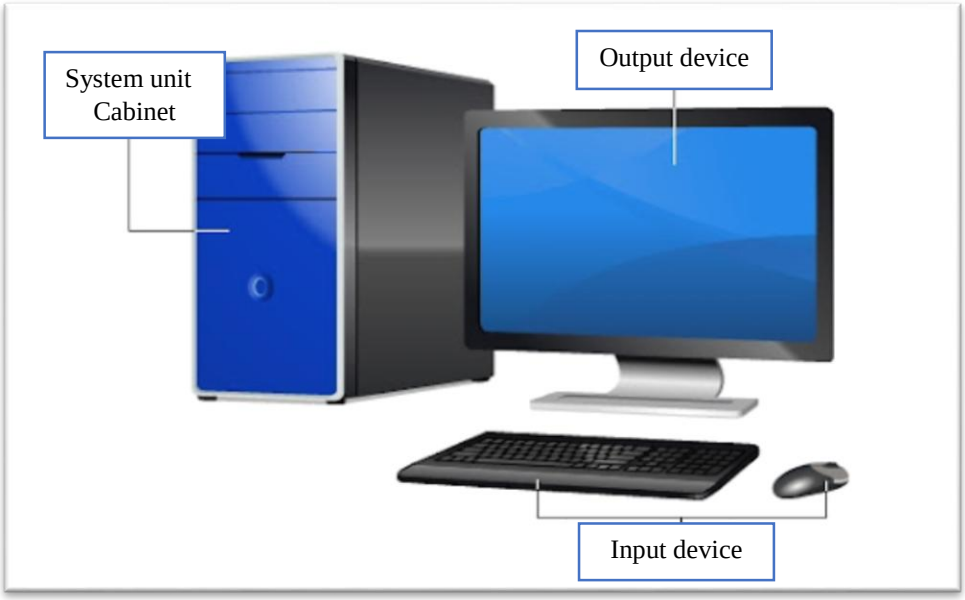
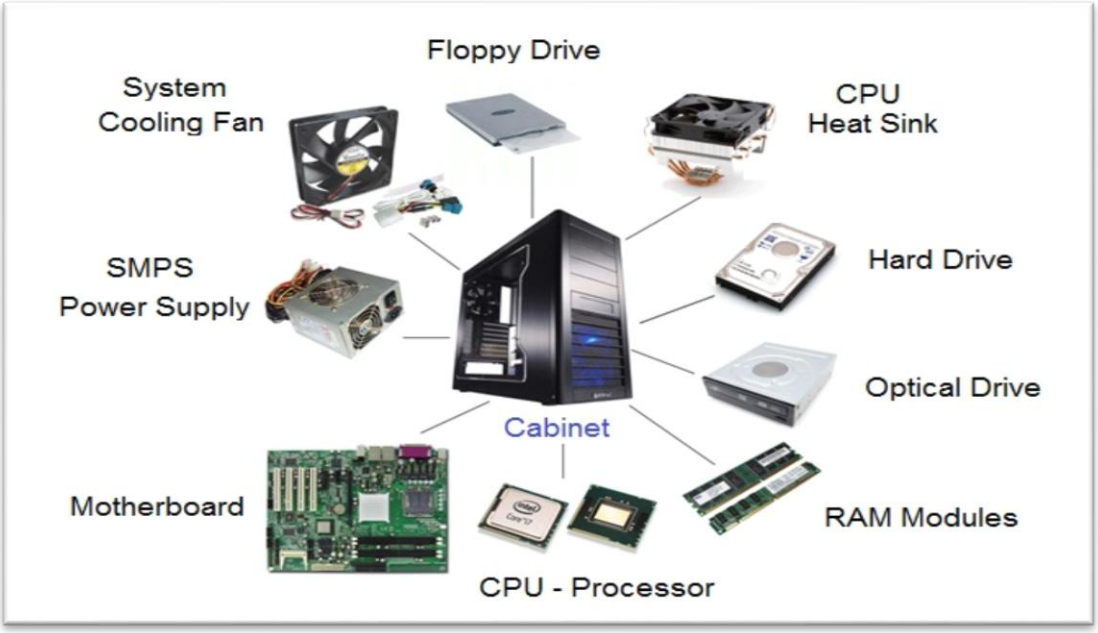
1-1 Introduction

Computer is a device capable of performing computations and making logical decisions at speeds millions and even billions of times faster than human beings.





- Computers process data under the control of sets of instructions that called **computer programs.**
- **Programming** is the process of writing instructions for a computer in a certain order to solve a problem.
- The computer programs that run on a computer are referred to as **Software (SW).** While the hard component of it is called **Hardware (HW).**
- Developing new software requires written lists of instructions for a computer to execute. Programmers rarely write in the **language** directly understood by a computer.

1-2 Programing Languages Types

Programming languages are often categorized into three main types based on their level of abstraction from the computer's hardware:

a- Low-Level Languages:

These languages are very close to the machine's instruction set, offering direct control over hardware but requiring a deep understanding of computer architecture.

- **Machine Language:** This is the most fundamental language, consisting of binary code (0s and 1s) directly understood by the CPU. It is extremely difficult for humans to read and write.
- **Assembly Language:** This uses symbolic representations (mnemonics) for machine instructions, making it slightly more readable than machine code. An assembler converts assembly code into machine code.

b- Middle-Level Languages:

This category is sometimes used to describe languages that offer a balance between low-level control and high-level abstraction.

Example: C is often considered a middle-level language as it provides features for direct memory manipulation while also supporting structured programming constructs.

c- High-Level Languages:

These languages are designed to be more human-readable and abstract away the complexities of the underlying hardware. They are easier to learn, write, and debug.

- **Examples:** Python, Java, JavaScript, C++, and many others fall into this category. They require a compiler or an interpreter to translate the code into machine-executable instructions.

While these three categories provide a foundational classification, high-level languages are further categorized based on their programming paradigms, such as:

- **Procedural Languages:**

  Focus on a sequence of steps or procedures to achieve a result (e.g., C, FORTRAN).

- **Object-Oriented Languages (OOP):**

  Organize programs around objects that contain both data and methods (e.g., Java, C++, Python).

- **Functional Languages:**

  Treat computation as the evaluation of mathematical functions (e.g., Lisp, Haskell).

- **Scripting Languages:**
  Often used for automating tasks and web development, typically interpreted (e.g., Python, JavaScript, Ruby).

1-3 C++ programming language

For the last couple of decades, the C programming language has been widely accepted for all applications, and is perhaps the most powerful of structured programming languages. Now, C++ has the status of a structured programming language with object oriented programming (OOP). C++ has become quite popular due to the following reasons:

1. It supports all features of both structured programming and OOP.
2. C++ focuses on function and class templates for handling data types.

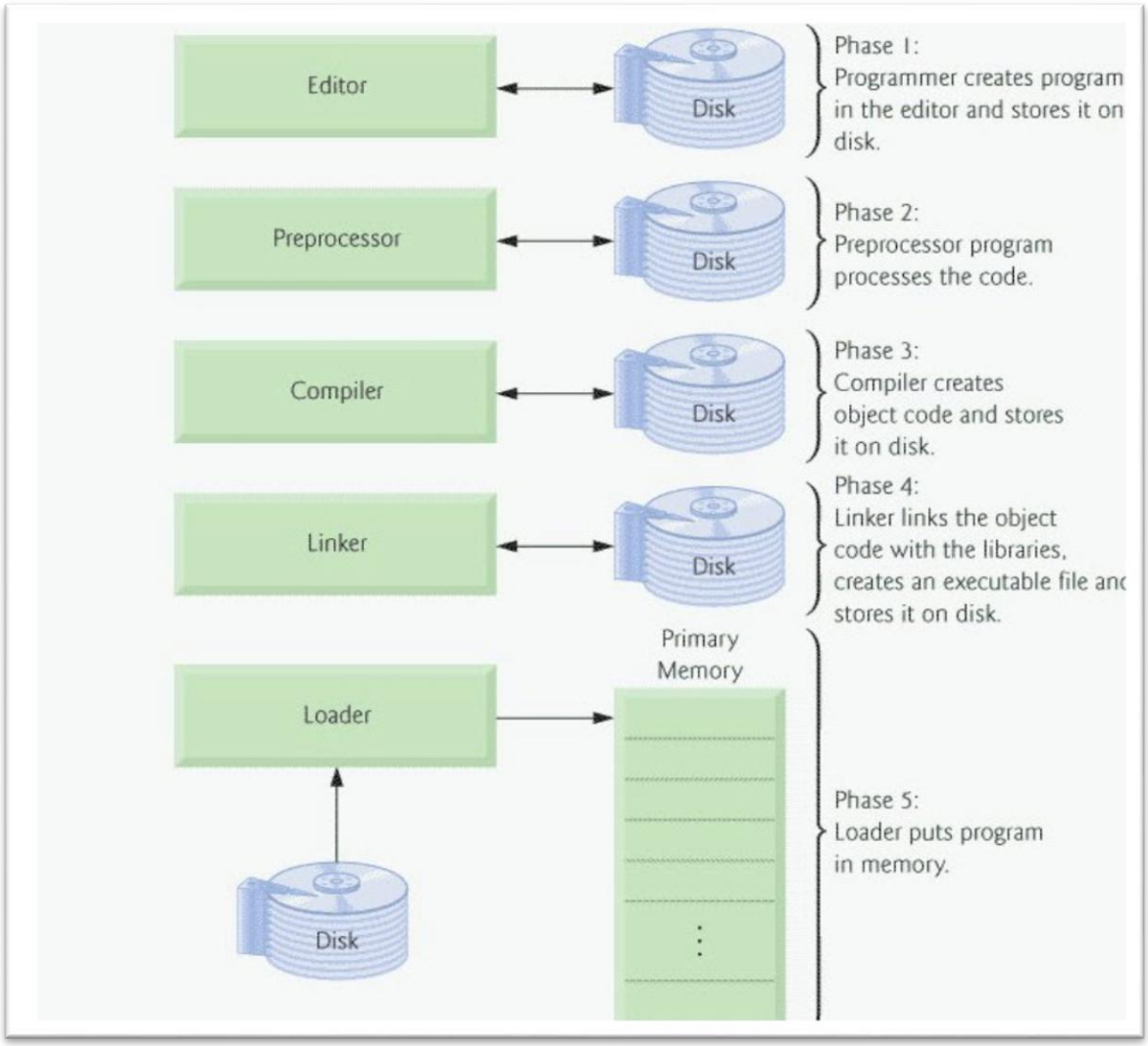
1-4 C++ program Development Process (PDP)

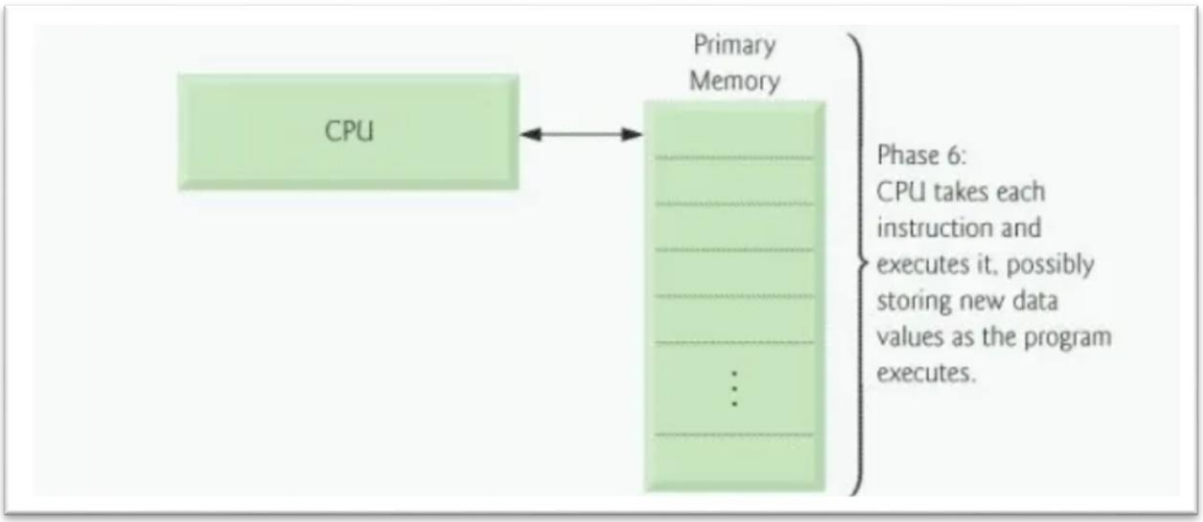
C++ programs typically go through six phases before they can be executed. These phases are:

1. **Edit:** The programmer types a C++ source program, and makes correction, if necessary. Then file is stored in disk with extension (.cpp).
2. **Pre-Processor:** Pre-processing is accomplished by the pre-processor before compilation, which includes some substitution of files and other directories to be include with the source file (the pre-processor handles instructions like #include and #define).
3. **Compilation:** The compiler translates the pre-processed C++ source code into machine-readable object code. The compiler can understand your code and translate it into machine language only if your code is in the proper syntax for that programming language. If there is a syntax error, then the compiler cannot translate your code into machine language instructions, and instead will call your attention to the syntax errors. Thus, in a sense, the compiler acts as a spell checker and grammar checker.

4. **Linking:** The linker combines the compiled object code with necessary library functions and other object files to create an executable program.
5. **Loading:** The operating system's loader places the executable program into main memory (RAM), preparing it for execution.
6. **CPU:** Executes the program, residing in memory and performs the tasks defined by the C++ code.

These steps are introduced in the figure below:


Editor
Disk
Phase 1:
Programmer creates program in the editor and stores it on disk.
Preprocessor
Disk
Phase 2:
Preprocessor program processes the code.
Compiler
Disk
Phase 3:
Compiler creates object code and stores it on disk.
Linker
Disk
Phase 4:
Linker links the object code with the libraries, creates an executable file and stores it on disk.
Primary Memory
Loader
Disk
Phase 5:
Loader puts program in memory.


Primary Memory
CPU
Phase 6:
CPU takes each instruction and executes it, possibly storing new data values as the program executes.

1-5 ALGORITHMS

As stated earlier an algorithm can be defined as a finite sequence of effect statements to solve a problem. An effective statement is a clear, unambiguous instruction that can be carried out. Thus an algorithm should special the action to be executed and the order in which these actions are to be executed.

What are the ALGORITHM properties?

•**Finiteness:** the algorithm must terminate a finite number of steps.

• **Non-ambiguity:** each step must be precisely defined. At the completion of each step, the nest step should be uniquely determined.

• **Effectiveness:** the algorithm should solve the problem in a reasonable amount of time.

Example 1: **Develop an algorithm that inputs a series of number and output their average.**

A computer algorithm can only carry out simple instruction like:

• "Read a number".
• "Add a number to anther number".
• "Output a number".

Thus an algorithm is:

1. Carry out initialization required (variables and their types definition).
2. Read first number.
3. While the length of series is not complete do
4. begin
    5. Add the number to the accumulated sum.
    6. increment the count of numbers entered.
    7. Read next number.
8. End
9. Evaluate the average.

Example 2: Develop an algorithm that allows the user to enter the count of numbers in a list followed by these numbers. The algorithm should find and output the minimum and the maximum numbers in the list.

An algorithm for this might be:

• Initialize.

• Get count of numbers.

• Enter numbers and find maximum and minimum.

• Output result.

The user might enter zero for the count. To deal with this case the above general case can be extended as follows to be an algorithm:

1. Initialize the require variables.
2. Get count of numbers.
3. If count is zero then exit.
4. Otherwise begin.
    5. Enter numbers.
    6. Find max and min.
    7. Output result.
8. End.